

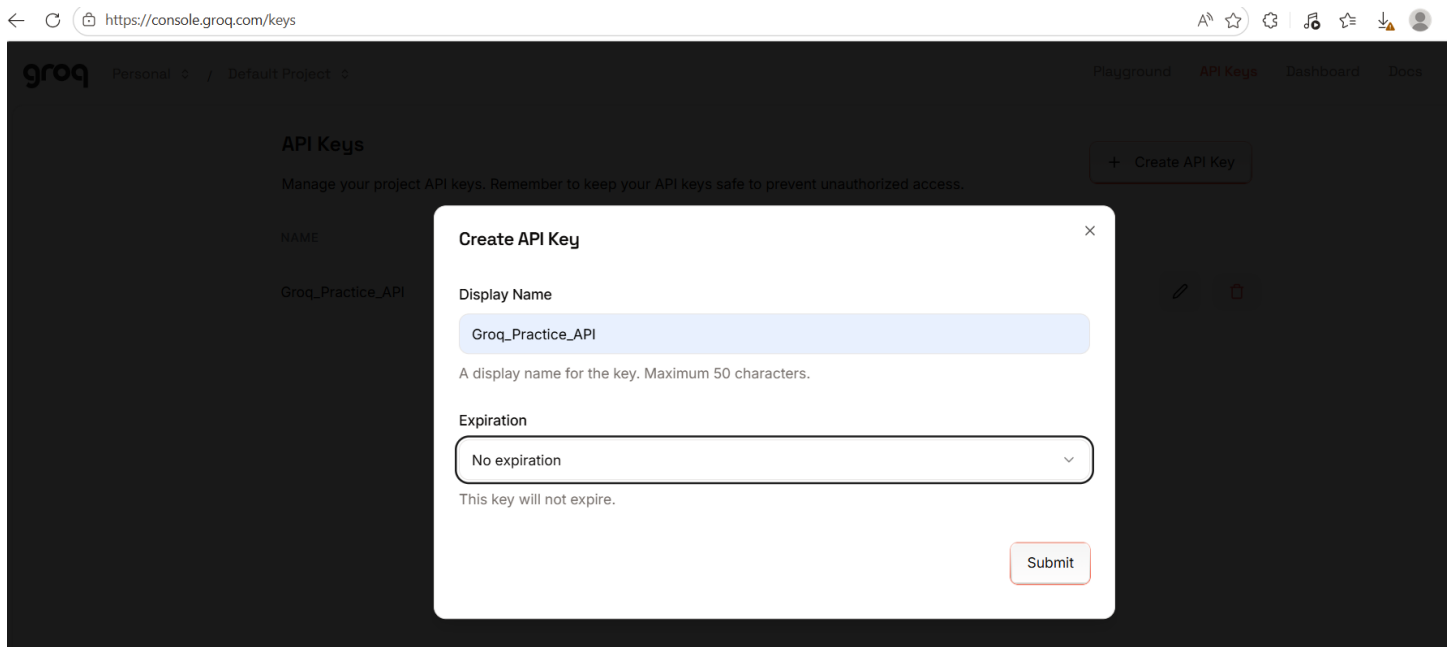
LLM API – JSON INTEGRATION

Step 1: Getting API Endpoint

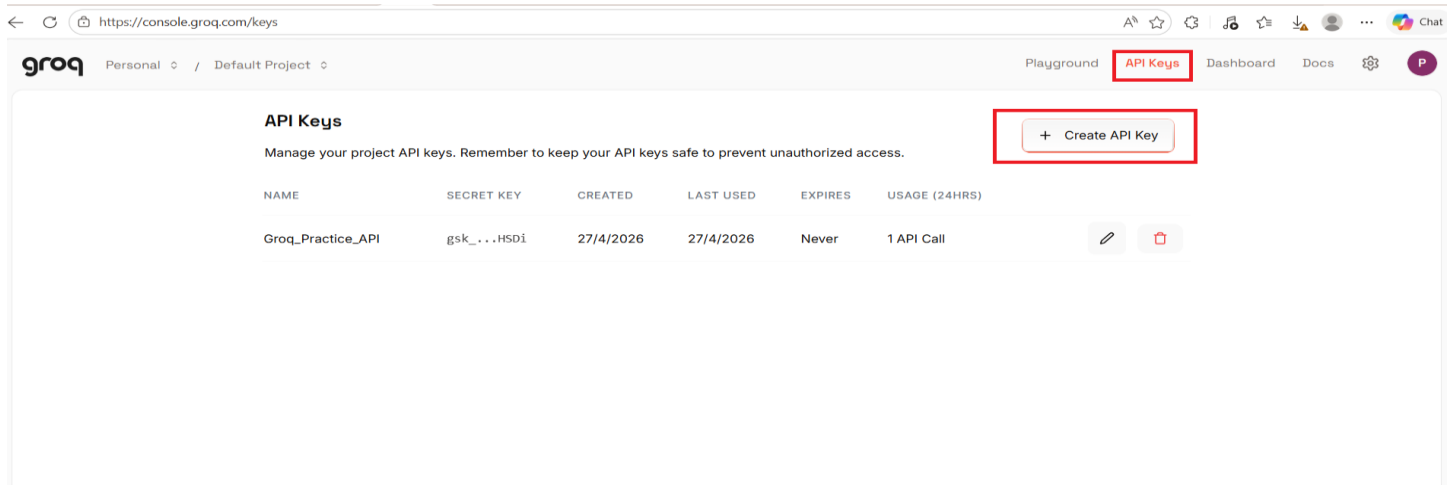
- 1) Login to Groq: [API Keys - GroqCloud](#)
- 2) Create API Keys: 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
- 3) Click on Playground at the right top
- 4) Select curl for the dropdown
- 5) Copy the url

<https://api.groq.com/openai/v1/chat/completions>

1)

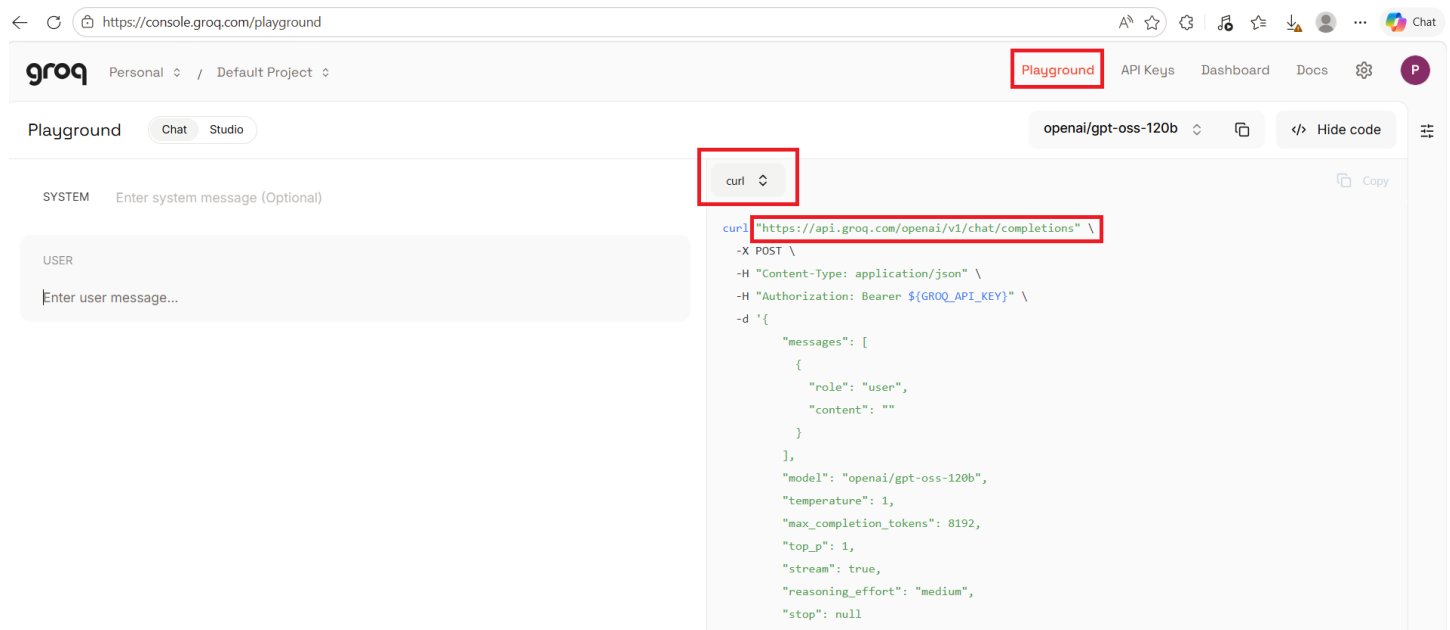


2)



3) Copy the url

<https://api.groq.com/openai/v1/chat/completions>



---Forming request body---

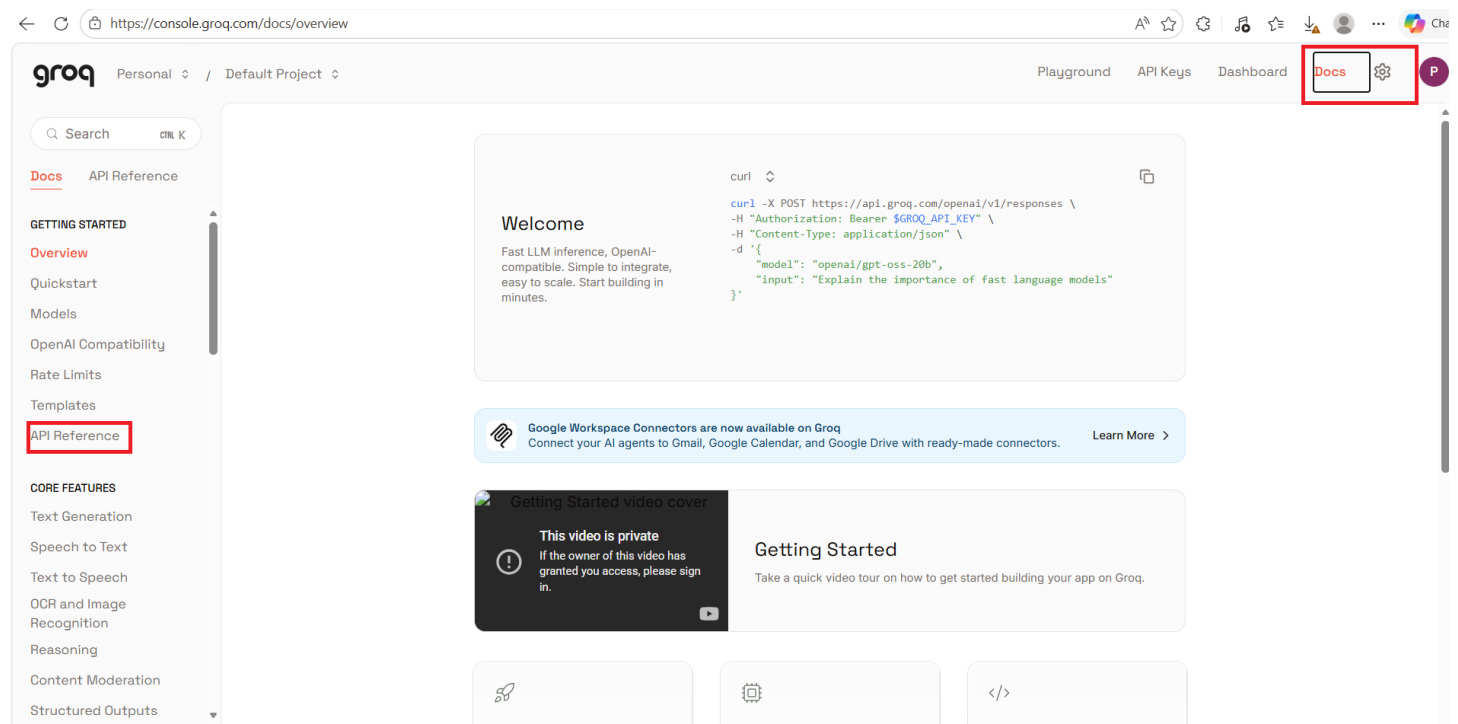
- 1)Login to Groq
- 2)Click on Docs menu
- 3)Click on API reference

Mandatory Parameters -- messages ,model

Context (System message /role -system)

Instructions (user input/role-user)

Output (Assitant message/Assistance) --Expected output (Request body)



- Mandatory Parameters -- messages ,model

The screenshot shows the Groq API Reference page for the 'Create chat completion' endpoint. The left sidebar contains a search bar and a list of endpoints under the 'Chat' section, with 'Create chat completion' highlighted. The main content area shows the endpoint details: POST https://api.groq.com/openai/v1/chat/completions. It describes the endpoint's purpose and lists the request body parameters. The 'messages' parameter is highlighted with a red box, showing it is an array of objects with 'role' and 'content' fields. The 'model' parameter is also highlighted with a red box, indicating it is a string representing the model ID. The 'citation_options' and 'compound_custom' parameters are also listed. On the right, there is a 'curl' command and an 'Example Response' in JSON format.

Create chat completion

POST https://api.groq.com/openai/v1/chat/completions

Creates a model response for the given chat conversation.

Request Body

messages array **Required**
A list of messages comprising the conversation so far.
Show possible types

model string **Required**
ID of the model to use. For details on which models are compatible with the Chat API, see available [models](#)

citation_options string or null **Optional** Defaults to enabled
Allowed values: enabled, disabled
Whether to enable citations in the response. When enabled, the model will include citations for information retrieved from provided documents or web searches.

compound_custom object or null **Optional**
Custom configuration of models and tools for Compound.
Show properties

curl

```
curl https://api.groq.com/openai/v1/chat/completions -s \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $GROQ_API_KEY" \
-d '{
  "model": "llama-3.3-70b-versatile",
  "messages": [
    {
      "role": "user",
      "content": "Explain the importance of fast language model"
    }
  ]
}'
```

Example Response

```
{
  "id": "chatcmpl-f51b2cd2-bef7-417e-964e-a08f0b513c22",
  "object": "chat.completion",
  "created": 1730241104,
  "model": "openai/gpt-oss-20b",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Fast language models have gained significant"
      },
      "logprobs": null,
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 10,
    "completion_tokens": 10,
    "total_tokens": 20
  }
}
```

Context is going to be a (System message /role -system)

This screenshot shows the same Groq API Reference page, but with the 'messages' parameter expanded to show its structure. The 'System message' object is highlighted with a red box, indicating its role in the conversation. The 'User message', 'Assistant message', 'Tool message', and 'Function message' (marked as deprecated) are also listed. The 'curl' command and 'Example Response' remain the same as in the previous screenshot.

Create chat completion

POST https://api.groq.com/openai/v1/chat/completions

Creates a model response for the given chat conversation.

Request Body

messages array **Required**
A list of messages comprising the conversation so far.
Show possible types

- System message** object
Show properties
- User message** object
Show properties
- Assistant message** object
Show properties
- Tool message** object
Show properties
- Function message** **Deprecated** object
Show properties

curl

```
curl https://api.groq.com/openai/v1/chat/completions -s \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $GROQ_API_KEY" \
-d '{
  "model": "llama-3.3-70b-versatile",
  "messages": [
    {
      "role": "user",
      "content": "Explain the importance of fast language model"
    }
  ]
}'
```

Example Response

```
{
  "id": "chatcmpl-f51b2cd2-bef7-417e-964e-a08f0b513c22",
  "object": "chat.completion",
  "created": 1730241104,
  "model": "openai/gpt-oss-20b",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Fast language models have gained significant"
      },
      "logprobs": null,
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 10,
    "completion_tokens": 10,
    "total_tokens": 20
  }
}
```

https://console.groq.com/playground?model=openai/gpt-oss-120b

Personal / Default Project

Playground Chat Studio

openai/gpt-oss-120b

SYSTEM Enter system message (Optional)

USER Enter user message...

```
curl -X POST \
-H "Content-Type: application/json" \
-H "Authorization: Bearer ${GROQ_API_KEY}" \
-d '{
  "messages": [
    {
      "role": "user",
      "content": ""
    }
  ],
  "model": "openai/gpt-oss-120b",
  "temperature": 1,
  "max_completion_tokens": 8192,
  "top_p": 1,
  "stream": true,
  "reasoning_effort": "medium",
  "stop": null
}'
```

Go to Postman:

GenAI-March

POST S02-GenerateTestcasesFromUserStories

POST https://api.groq.com/openai/v1/chat/completions

Body

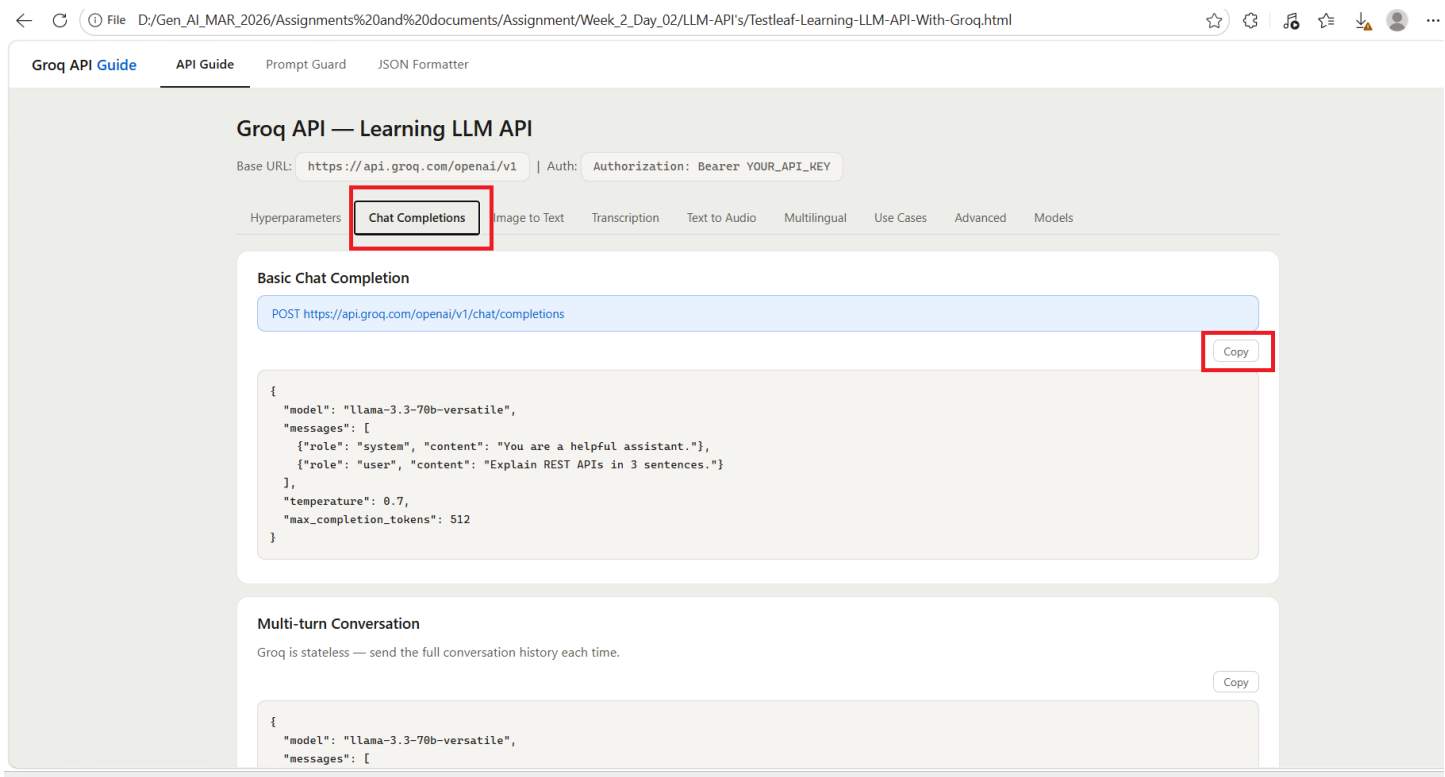
```
1 {
2   "model": "openai/gpt-oss-120b",
3   "messages": [
4     {
5       "role": "system",
6       "content": "You are a Senior QA Analyst experienced in converting Agile user stories into high-quality, review-ready test cases."
7     },
8     {
9       "role": "user",
10      "content": "I - INSTRUCTION\nConvert the provided User Stories and Acceptance Criteria into detailed, testable Test Cases.\n\nMANDATORY RULES:\n1) Derive test cases ONLY from the given user stories and acceptance criteria.\n2) Do NOT add assumptions or new requirements.\n3) Cover positive, negative, and validation scenarios.\n4) Write test cases from an end-user perspective.\n5) Do NOT include implementation or technical details.\n\nCONTEXT\nThese test cases will be used by QA teams for manual and automation testing in an enterprise healthcare application.\n\nPERSONA\nYou are a Senior QA Analyst experienced in converting Agile user stories into..."
9     }
10  ]
11 }
```

Body Cookies 1 Headers 21 Test Results 200 OK 6.51 s 2.24 KB

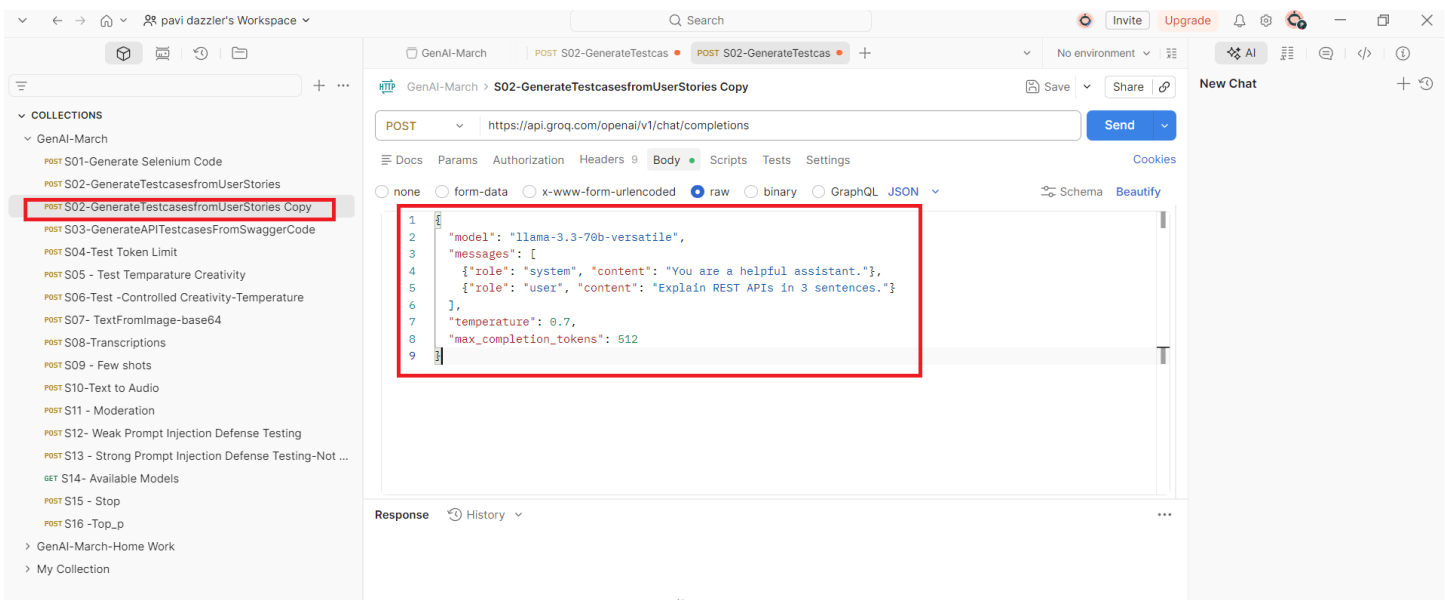
```
1 {
2   "id": "chatcmpl-8ffcid1-696c-4f3c-878b-a587154862af",
3   "object": "chat.completion",
4   "created": 1777296799,
5   "model": "meta-llama/llama-4-scout-17b-16e-instruct",
6   "choices": [
7     {
8       "index": 0,
9       "message": {
10        "role": "assistant",
11        "content": "I - INSTRUCTION\nConvert the provided User Stories and Acceptance Criteria into detailed, testable Test Cases.\n\nMANDATORY RULES:\n1) Derive test cases ONLY from the given user stories and acceptance criteria.\n2) Do NOT add assumptions or new requirements.\n3) Cover positive, negative, and validation scenarios.\n4) Write test cases from an end-user perspective.\n5) Do NOT include implementation or technical details.\n\nCONTEXT\nThese test cases will be used by QA teams for manual and automation testing in an enterprise healthcare application.\n\nPERSONA\nYou are a Senior QA Analyst experienced in converting Agile user stories into..."
12       },
13       "logprobs": null,
14       "finish_reason": "stop"
15     }
16   ]
17 }
```

Go to HTML File generated tool:

[Testleaf-Learning LLM API with Groq](#)



Navigate Postman: Duplicate any of the file
Delete present JSON and Paste the chat over the file:



Then click beautify and paste this below:

I-Instructions (user input/user))

Generate testcases for Payment (Fund Transfer) module

Generate Only for UPI and IMPS

[IMPORTANT]-Generate only P1 and P2 testcases only functional as category

[CRITICAL]-Generate only for Happy Path scenarios E2E testcase

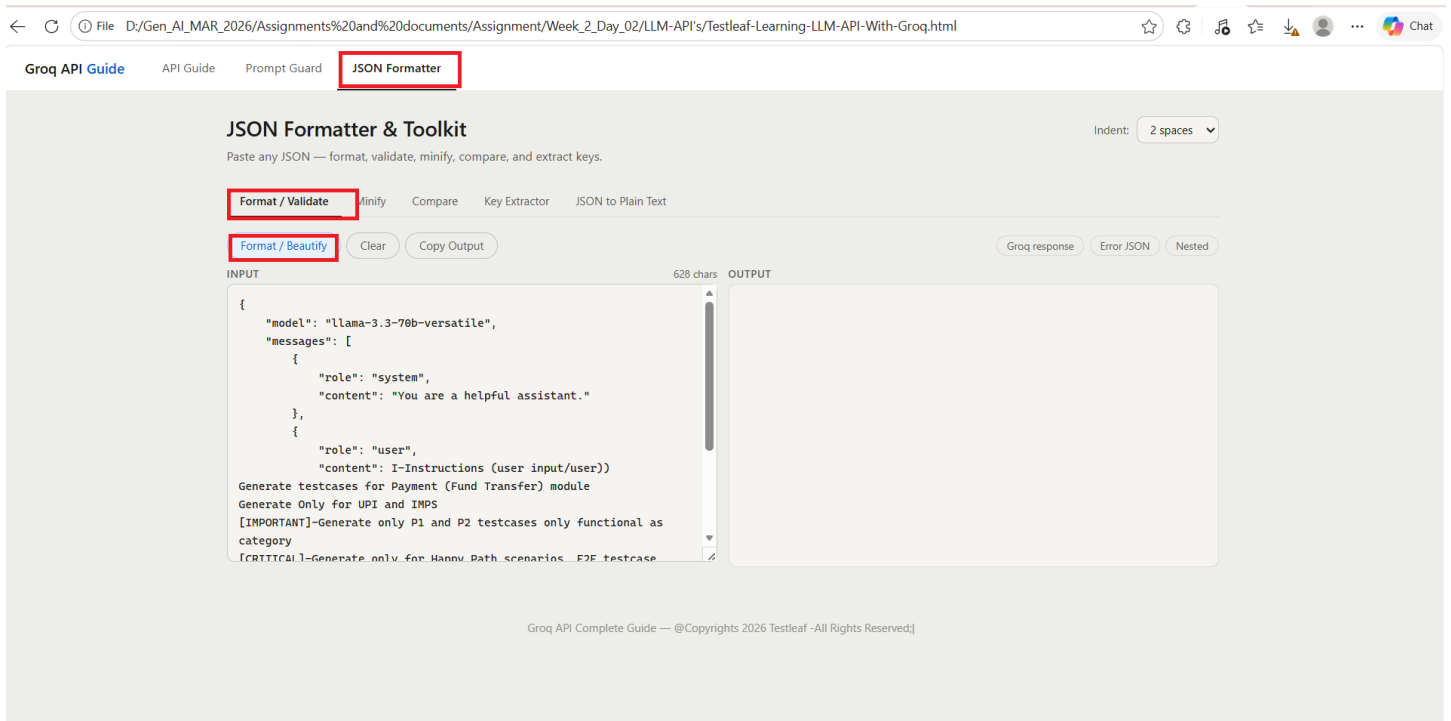
[IMPORTANT] -Generate ONLY total of 5 testcases each for UPI and IMPS

In Postman tool:

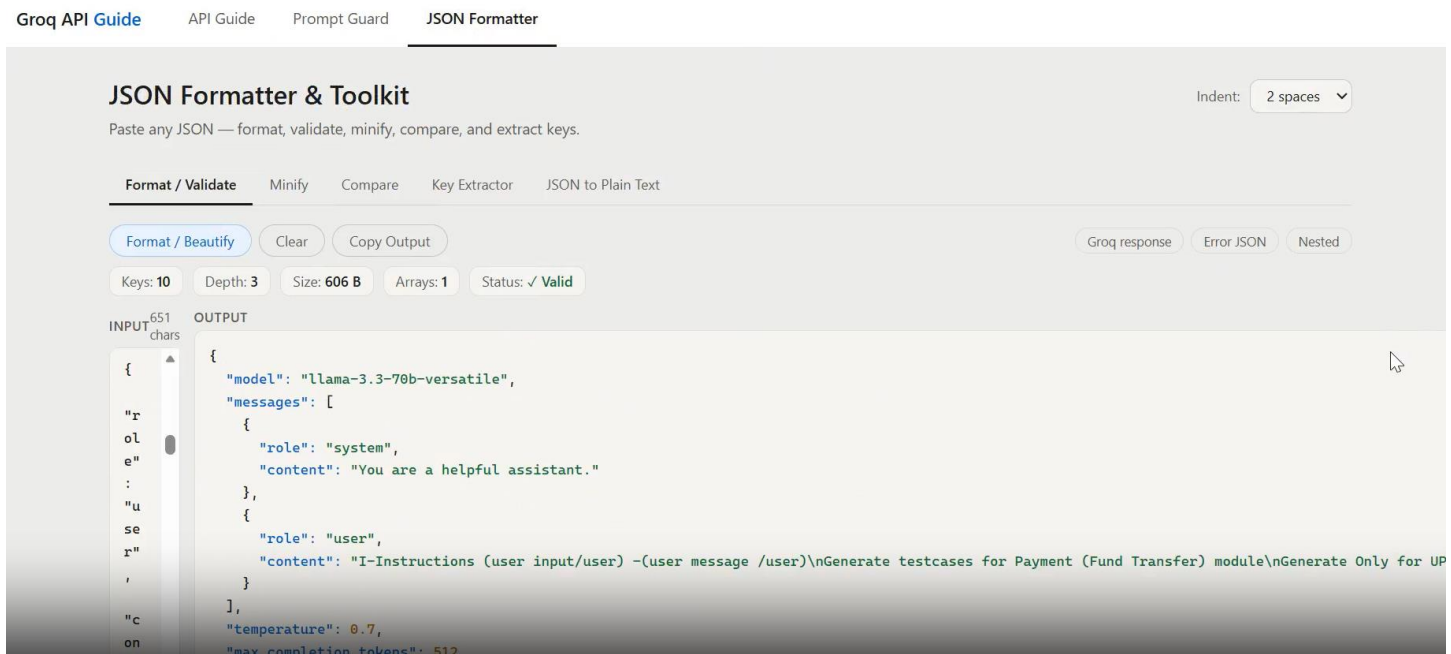
```
{  
  
  "model": "llama-3.3-70b-versatile",  
  
  "messages": [  
  
    {  
  
      "role": "system",  
  
      "content": "You are a helpful assistant." },  
  
    {  
  
      "role": "user",  
  
      "content": I-Instructions (user input/user))  
  
      Generate testcases for Payment (Fund Transfer) module  
  
      Generate Only for UPI and IMPS  
  
      [IMPORTANT]-Generate only P1 and P2 testcases only functional as category  
  
      [CRITICAL]-Generate only for Happy Path scenarios E2E testcase  
  
      [IMPORTANT] -Generate ONLY total of 5 testcases each for UPI and IMPS  
  
    } ],  
  
    "temperature": 0.7,  
  
    "max_completion_tokens": 512  
  
  }  
}
```

In Testleaf HTML tool which is Utility Tool

The screenshot shows the Testleaf JSON Formatter & Toolkit web application. The browser address bar shows the URL: `D:/Gen_AI_MAR_2026/Assignments%20and%20documents/Assignment/Week_2_Day_02/LLM-API's/Testleaf-Learning-LLM-API-With-Groq.html`. The application has a navigation bar with links: [Groq API Guide](#), [API Guide](#), [Prompt Guard](#), and [JSON Formatter](#) (which is the active tab). The main heading is **JSON Formatter & Toolkit**, with a subtext: "Paste any JSON — format, validate, minify, compare, and extract keys." On the right, there is an "Indent:" dropdown menu set to "2 spaces". Below the heading, there are tabs: **Format / Validate**, [Minify](#), [Compare](#), [Key Extractor](#), and [JSON to Plain Text](#). Under the **Format / Validate** tab, there are buttons: [Format / Beautify](#) (highlighted in blue), [Clear](#), and [Copy Output](#). To the right of these buttons are three radio buttons: [Groq response](#), [Error JSON](#), and [Nested](#). The interface is divided into two main sections: **INPUT** and **OUTPUT**. The **INPUT** section has a text area with the placeholder "Paste JSON here ...". The **OUTPUT** section is currently empty. At the bottom of the page, there is a footer: "Groq API Complete Guide — ©Copyrights 2026 Testleaf -All Rights Reserved|".



Click and Paste and copy output:



Chat gpt qwery mistake JSON format:

This is the json format seems mistake.

Could you pls beautify it

{

```

"model": "llama-3.3-70b-versatile",

"messages": [

  {

    "role": "system",

    "content": "You are a helpful assistant."

  },

  {

    "role": "user",

    "content": I-Instructions (user input/user))

```

Generate testcases for Payment (Fund Transfer) module

Generate Only for UPI and IMPS

[IMPORTANT]-Generate only P1 and P2 testcases only functional as category

[CRITICAL]-Generate only for Happy Path scenarios E2E testcase

[IMPORTANT] -Generate ONLY total of 5 testcases each for UPI and IMPS

```

    }

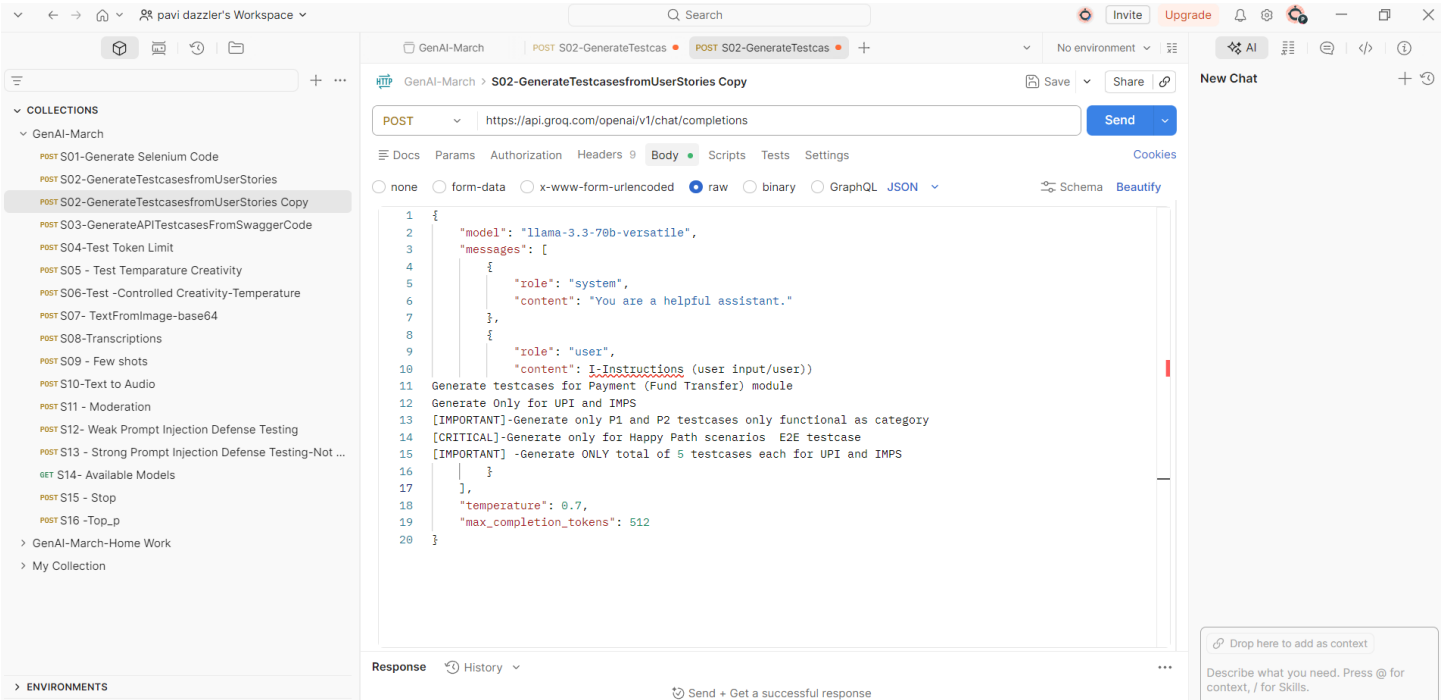
  ],

  "temperature": 0.7,

  "max_completion_tokens": 512

}

```



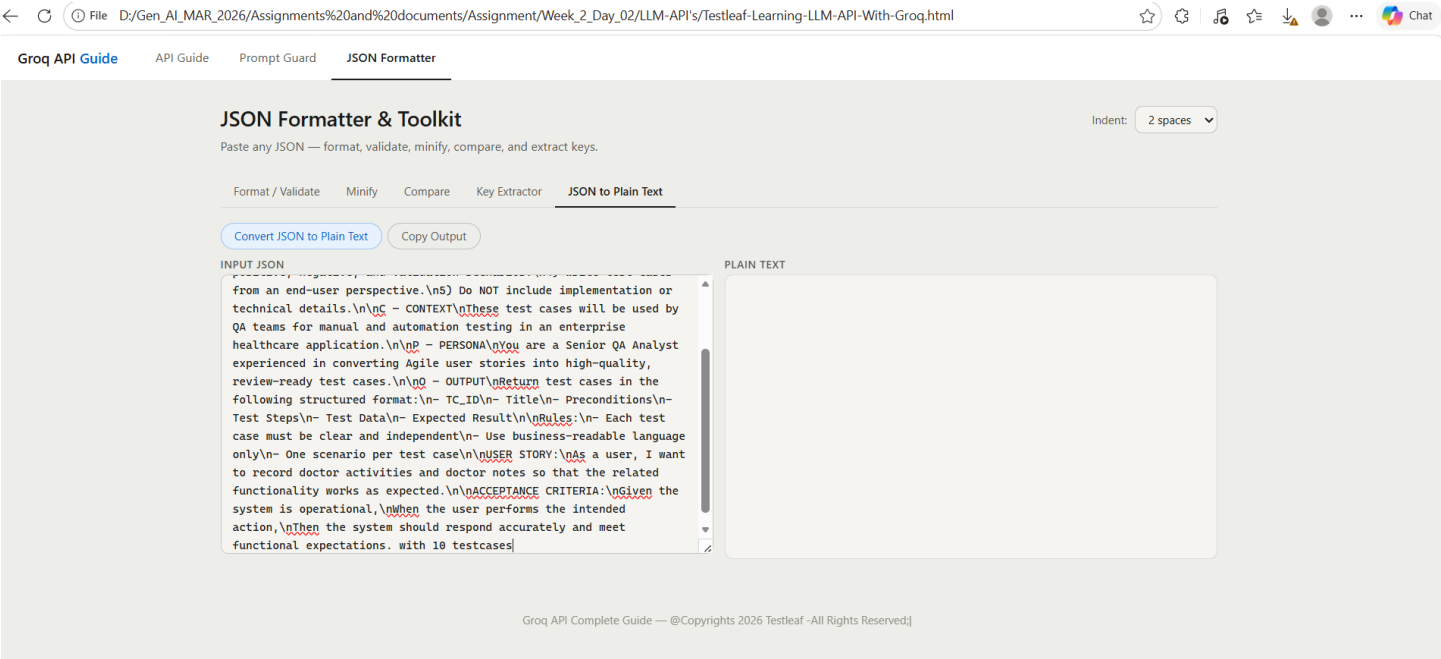
After beautify with GPT Answer:

```
{
  "model": "llama-3.3-70b-versatile",
  "messages": [
    {
      "role": "system",
      "content": "You are a helpful assistant."
    },
    {
      "role": "user",
      "content": "Instructions (user input):\nGenerate testcases for Payment (Fund Transfer) module.\nGenerate only for UPI and IMPS.\n[IMPORTANT] Generate only P1 and P2 testcases (functional category only).\n[CRITICAL] Generate only Happy Path scenarios (E2E testcases).\n[IMPORTANT] Generate ONLY a total of 5 testcases each for UPI and IMPS."
    }
  ],
  "temperature": 0.7,
  "max_completion_tokens": 512
}
```

And paste it in the postman post file

The screenshot shows the Postman interface with a workspace named 'pavi dazzler's Workspace'. The active collection is 'GenAI-March', and the selected request is 'S02-GenerateTestcasesfromUserStories Copy'. The request is a POST to 'https://api.groq.com/openai/v1/chat/completions' with a 'raw' body containing the JSON payload from the previous block. The response is a 200 OK status, and the body is displayed in JSON format, showing the chat completion details including the model name, messages, and the generated response.

Approach: Postmn S02-GenerateTestcasesfromUserStories:
Navigate to utility test leaf:



Copy ouput and paste:

I — INSTRUCTION

Convert the provided User Stories and Acceptance Criteria into detailed, testable Test Cases.

MANDATORY RULES:

- 1) Derive test cases ONLY from the given user stories and acceptance criteria.
- 2) Do NOT add assumptions or new requirements.
- 3) Cover positive, negative, and validation scenarios.
- 4) Write test cases from an end-user perspective.
- 5) Do NOT include implementation or technical details.

C — CONTEXT

These test cases will be used by QA teams for manual and automation testing in an enterprise healthcare application.

P — PERSONA

You are a Senior QA Analyst experienced in converting Agile user stories into high-quality, review-ready test cases.

O — OUTPUT

Return test cases in the following structured format:

- TC_ID
- Title
- Preconditions
- Test Steps
- Test Data
- Expected Result

Rules:

- Each test case must be clear and independent
- Use business-readable language only
- One scenario per test case

USER STORY:

As a user, I want to record doctor activities and doctor notes so that the related functionality works as expected.

ACCEPTANCE CRITERIA:

Given the system is operational,

When the user performs the intended action,

Then the system should respond accurately and meet functional expectations. with 10 testcases

Add API Key to the collections:

Go to the postman collections

Click on Authorization

Auth Type Dropdown --Select Bearer Token

Copy paste the Key: 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

Response

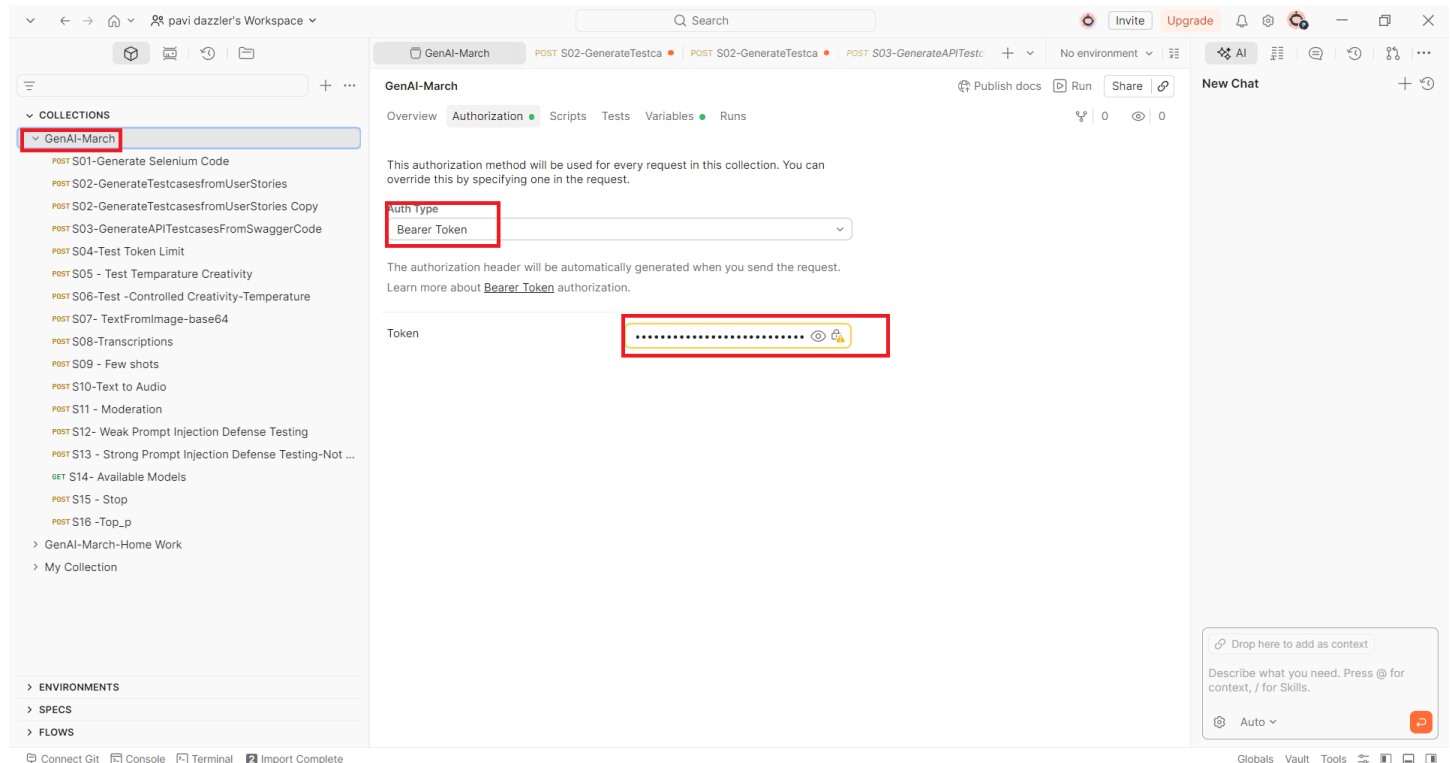
messages --role :assistant-content --output generated

prompt_tokens --input tokens

completion_tokens --output tokens

total_tokens --prompt_tokens+ completion_tokens

"finish_reason": "stop" --entire output is



Project Assignment:

Step 1: Access POST S02-GenerateTestcasesfromUserStories:

The screenshot shows a REST client interface with a sidebar on the left containing a list of collections. The main panel displays a POST request to the URL `https://api.groq.com/openai/v1/chat/completions`. The request body is in JSON format, containing a system message and a user message. The system message instructs the model to act as a Senior QA Analyst and generate high-quality, review-ready test cases from user stories. The user message provides a specific instruction to convert provided user stories and acceptance criteria into detailed, testable test cases, following mandatory rules: derive test cases only from given user stories and acceptance criteria, do not add assumptions or new requirements, cover positive, negative, and validation scenarios, and write test cases from an end-user perspective. The response is a 200 OK status, indicating a successful request. The response body is a JSON object containing the generated test cases.

```
POST https://api.groq.com/openai/v1/chat/completions

{
  "model": "openai/gpt-oss-120b",
  "messages": [
    {
      "role": "system",
      "content": "You are a Senior QA Analyst experienced in converting Agile user stories into high-quality, review-ready test cases."
    },
    {
      "role": "user",
      "content": "I - INSTRUCTION\nConvert the provided User Stories and Acceptance Criteria into detailed, testable Test Cases.\nMANDATORY RULES:\n1) Derive test cases ONLY from the given user stories and acceptance criteria.\n2) Do NOT add assumptions or new requirements.\n3) Cover positive, negative, and validation scenarios.\n4) Write test cases from an end-user perspective.\n5) Do NOT include implementation or technical details.\nCONTEXT\nThese test cases will be used by QA teams for manual and automation testing in an enterprise healthcare application.\nPERSONA\nYou are a Senior QA Analyst experienced in converting Agile user stories into high-quality, review-ready test cases.\nOUTPUT\nReturn test cases in the following structured format:\n- TC_ID\n- Title\n- Preconditions\n- Test Steps\n- Test Data\n- Expected Result\nEach test case must be clear and independent. Use business-readable"
    }
  ]
}
```

```
{
  "id": "chatcmpl-93600302-e2fb-4763-904f-e29210477318",
  "object": "chat.completion",
  "created": 1777304138,
  "model": "openai/gpt-oss-120b",
  "choices": [
    {
      "index": 0,
      "message": {

```

The screenshot shows the same REST client interface as the previous one, but with the response body highlighted. The response body is a JSON object containing the generated test cases. The response is a 200 OK status, indicating a successful request. The response body is a JSON object containing the generated test cases.

```
high-quality, review-ready test cases."
  },
  {
    "role": "user",
    "content": "I - INSTRUCTION\nConvert the provided User Stories and Acceptance Criteria into detailed, testable Test Cases.\nMANDATORY RULES:\n1) Derive test cases ONLY from the given user stories and acceptance criteria.\n2) Do NOT add assumptions or new requirements.\n3) Cover positive, negative, and validation scenarios.\n4) Write test cases from an end-user perspective.\n5) Do NOT include implementation or technical details.\nCONTEXT\nThese test cases will be used by QA teams for manual and automation testing in an enterprise healthcare application.\nPERSONA\nYou are a Senior QA Analyst experienced in converting Agile user stories into high-quality, review-ready test cases.\nOUTPUT\nReturn test cases in the following structured format:\n- TC_ID\n- Title\n- Preconditions\n- Test Steps\n- Test Data\n- Expected Result\nEach test case must be clear and independent. Use business-readable"
  }
]
}
```

```
{
  "id": "chatcmpl-93600302-e2fb-4763-904f-e29210477318",
  "object": "chat.completion",
  "created": 1777304138,
  "model": "openai/gpt-oss-120b",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "**Test Cases - Recording Doctor Activities & Doctor Notes** \n\n| TC_ID | Title | Preconditions | Test Steps | Test Data | Expected Result |\n|-----|-----|-----|-----|-----|-----|\n| TC_001 | Create a new doctor activity with all required fields populated | • System is operational <br>• User is logged in with permission to record activities | 1. Navigate to **Doctor Activities** screen. <br>2. Click **Add New Activity**. <br>3. Fill in each mandatory field (e.g., Activity Type, Date, Patient ID, Description). <br>4. Click **Save**. | Activity Type: \"Consultation\" <br>Date: today's date (MM/DD/YYYY) <br>Patient ID: 123456 <br>Description: \"Routine follow-up visit.\" | New activity is saved and appears in the activity list with the entered details. A confirmation message \"Activity recorded successfully\" is displayed. |\n| TC_002 | Create a doctor note attached to an existing activity | • System is operational <br>• User is logged in with permission to add notes <br>• At least one doctor activity already exists | 1. Open an existing doctor activity from the list. <br>2. Click **Add Note**. <br>3. Enter note text. <br>4. Click **Save Note**. | Note Text: \"Patient reports improvement in symptoms.\" |
```

Inorder to generate JSON output as a plain text. Copy entire message and give to json converter utility test leaf

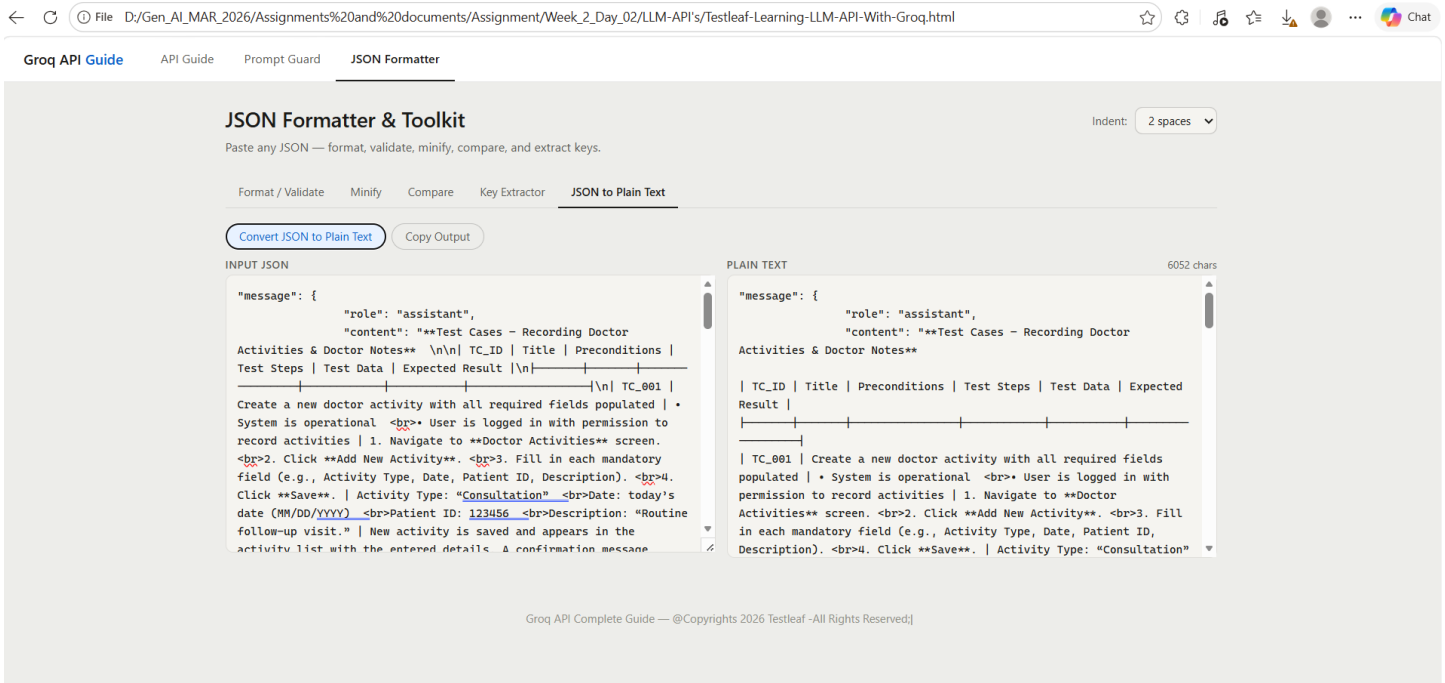
Take only CONTENT MESSAGE: "message": {

"role": "assistant",

"content": "***Test Cases – Recording Doctor Activities & Doctor Notes** \n\n| TC_ID | Title | Preconditions | Test Steps | Test Data | Expected Result |\n|-----|-----|-----|-----|-----|-----|\n| TC_001 | Create a new doctor activity with all required fields populated | • System is operational
• User is logged in with permission to record activities | 1. Navigate to **Doctor Activities** screen.
2. Click **Add New Activity**.
3. Fill in each mandatory field (e.g., Activity Type, Date, Patient ID, Description).
4. Click **Save**. | Activity Type: “Consultation”
Date: today’s date (MM/DD/YYYY)
Patient ID: 123456
Description: “Routine follow- up visit.” | New activity is saved and appears in the activity list with the entered details. A confirmation message “Activity recorded successfully” is displayed. |\n| TC_002 | Create a doctor note attached to an existing activity | • System is operational
• User is logged in with permission to add notes
• At least one doctor activity already exists | 1. Open an existing doctor activity from the list.
2. Click **Add Note**.
3. Enter note text.
4. Click **Save Note**. | Note Text: “Patient reports improvement in symptoms.” | The note is saved, displayed under the selected activity, and a confirmation “Note added successfully” appears. |\n| TC_003 | Verify that mandatory fields are validated when creating an activity | • System is operational
• User is logged in | 1. Navigate to **Doctor Activities**.
2. Click **Add New Activity**.
3. Leave all mandatory fields empty.
4. Click **Save**. | – | System blocks the save operation and shows appropriate validation messages for each missing required field (e.g., “Activity Type is required”, “Date is required”). |\n| TC_004 | Verify character limit enforcement on the activity description field | • System is operational
• User is logged in | 1. Open **Add New Activity** form.
2. Enter a description that exceeds the allowed length (e.g., 501 characters if limit is 500).
3. Attempt to save. | Description: 501- character string | System prevents saving and displays a message such as “Description cannot exceed 500 characters”. |\n| TC_005 | Attempt to save a doctor activity with a future date | • System is operational
• User is logged in | 1. Open **Add New Activity**.
2. Enter a future date (e.g., tomorrow).
3. Fill other mandatory fields with valid data.
4. Click **Save**. | Date: tomorrow’s date | System rejects the entry and shows a validation error “Date cannot be in the future”. |\n| TC_006 | Edit an existing doctor activity and change its description | • System is operational
• User is logged in
• At least one activity exists | 1. Locate an existing activity in the list.
2. Click **Edit**.
3. Modify the Description field.
4. Click **Save**. | New Description: “Updated after lab results.” | The activity is updated; the list now reflects the new description and a success message “Activity updated successfully” is shown. |\n| TC_007 | Delete a doctor note from an activity | • System is operational
• User is logged in
• An activity with at least one note exists | 1. Open the activity containing the note.
2. Click the **Delete** icon/button next to the note.
3. Confirm the deletion if prompted. | – | The note is removed from the activity, the list of notes no longer shows it, and a confirmation “Note deleted successfully” appears. |\n| TC_008 | Search for a doctor activity by Patient ID | • System is operational
• User is logged in | 1. Go to **Doctor Activities** screen.
2. Enter a known Patient ID in the search field.
3. Click **Search**. | Patient ID: 123456 | Only activities linked to Patient ID 123456 are displayed; all other records are excluded. |\n| TC_009 | Verify that a user without “record activity” permission cannot access the activity entry screen | • System is operational
• User is logged in with read- only role | 1. Attempt to navigate to **Doctor Activities → Add New Activity** (via URL or menu). | – | Access is denied; the system redirects to an “Access denied” page or shows a message “You do not have permission to record activities”. |\n| TC_010 | Verify system behavior when the network connection is lost during activity save | • System is operational
• User is logged in
• Network can be simulated to drop | 1. Open **Add New Activity** form.
2. Fill all required fields with valid data.
3. While clicking **Save**, simulate a network outage (e.g., disconnect internet).
4. Observe system response. | Valid activity data (any). | System displays an error message such as “Unable to save activity – please check your connection and try again” and does not create a partial record. The form remains populated so the user can retry after reconnection. |",

"reasoning": "The user wants 10 test cases derived from the user story and acceptance criteria. However the story is vague: record doctor activities and notes, and the AC is generic. We must still create 10 test cases covering positive, negative, validation. We cannot add assumptions, but we need some specifics. Since limited info, we can create generic test cases: creating a new doctor activity entry, editing, saving, required fields, character limits, duplicate entry, unauthorized user, view recorded notes, delete note, search, etc. Must stay within given story: \"record doctor activities and doctor notes\". We'll assume UI allows user to add activity, add note, save, view. We'll keep business language. Provide 10 test cases with IDs. Provide preconditions: system operational, user logged in, appropriate permissions. Provide steps, data, expected.\n\nMake sure not to add new requirements beyond what's implied. We can infer fields like activity type, date, description, note text. That's reasonable.\n\nLet's produce."

},



Copy the text and See the text based output here:

```
"message": {  
  "role": "assistant",  
  "content": "**Test Cases – Recording Doctor Activities & Doctor Notes**  
| TC_ID | Title | Preconditions | Test Steps | Test Data | Expected Result |  
|-----|-----|-----|-----|-----|-----|  
| TC_001 | Create a new doctor activity with all required fields populated | • System is operational <br>• User is logged in with permission to record activities | 1. Navigate to **Doctor Activities** screen. <br>2. Click **Add New Activity**. <br>3. Fill in each mandatory field (e.g., Activity Type, Date, Patient ID, Description). <br>4. Click **Save**. | Activity Type: "Consultation" <br>Date: today's date (MM/DD/YYYY) <br>Patient ID: 123456 <br>Description: "Routine follow-up visit." | New activity is saved and appears in the activity list with the entered details. A confirmation message "Activity recorded successfully" is displayed. |
```

```
| TC_ID | Title | Preconditions | Test Steps | Test Data | Expected Result |
```

```
|-----|-----|-----|-----|-----|-----|
```

```
| TC_001 | Create a new doctor activity with all required fields populated | • System is operational <br>• User is logged in with permission to record activities | 1. Navigate to **Doctor Activities** screen. <br>2. Click **Add New Activity**. <br>3. Fill in each mandatory field (e.g., Activity Type, Date, Patient ID, Description). <br>4. Click **Save**. | Activity Type: "Consultation" <br>Date: today's date (MM/DD/YYYY) <br>Patient ID: 123456 <br>Description: "Routine follow-up visit." | New activity is saved and appears in the activity list with the entered details. A confirmation message "Activity recorded successfully" is displayed. |
```

```
| TC_002 | Create a doctor note attached to an existing activity | • System is operational <br>• User is logged in with permission to add notes <br>• At least one doctor activity already exists | 1. Open an existing doctor activity from the list. <br>2. Click **Add Note**. <br>3. Enter note text. <br>4. Click **Save Note**. | Note Text: "Patient reports improvement in symptoms." | The note is saved, displayed under the selected activity, and a confirmation "Note added successfully" appears. |
```

```
| TC_003 | Verify that mandatory fields are validated when creating an activity | • System is operational <br>• User is logged in | 1. Navigate to **Doctor Activities**. <br>2. Click **Add New Activity**. <br>3. Leave all mandatory fields empty. <br>4. Click **Save**. | – | System blocks the save operation and shows appropriate validation messages for each missing required field (e.g., "Activity Type is required", "Date is required"). |
```

```
| TC_004 | Verify character limit enforcement on the activity description field | • System is operational <br>• User is logged in | 1. Open **Add New Activity** form. <br>2. Enter a description that exceeds the allowed
```


length (e.g., 501 characters if limit is 500).
3. Attempt to save. | Description: 501- character string | System prevents saving and displays a message such as “Description cannot exceed 500 characters”. |

| TC_005 | Attempt to save a doctor activity with a future date | • System is operational
• User is logged in
| 1. Open **Add New Activity**.
2. Enter a future date (e.g., tomorrow).
3. Fill other mandatory fields with valid data.
4. Click **Save**. | Date: tomorrow’s date | System rejects the entry and shows a validation error “Date cannot be in the future”. |

| TC_006 | Edit an existing doctor activity and change its description | • System is operational
• User is logged in
• At least one activity exists | 1. Locate an existing activity in the list.
2. Click **Edit**.
3. Modify the Description field.
4. Click **Save**. | New Description: “Updated after lab results.” | The activity is updated; the list now reflects the new description and a success message “Activity updated successfully” is shown. |

| TC_007 | Delete a doctor note from an activity | • System is operational
• User is logged in
• An activity with at least one note exists | 1. Open the activity containing the note.
2. Click the **Delete** icon/button next to the note.
3. Confirm the deletion if prompted. | – | The note is removed from the activity, the list of notes no longer shows it, and a confirmation “Note deleted successfully” appears. |

| TC_008 | Search for a doctor activity by Patient ID | • System is operational
• User is logged in | 1. Go to **Doctor Activities** screen.
2. Enter a known Patient ID in the search field.
3. Click **Search**. | Patient ID: 123456 | Only activities linked to Patient ID 123456 are displayed; all other records are excluded. |

| TC_009 | Verify that a user without “record activity” permission cannot access the activity entry screen | • System is operational
• User is logged in with read- only role | 1. Attempt to navigate to **Doctor Activities → Add New Activity** (via URL or menu). | – | Access is denied; the system redirects to an “Access denied” page or shows a message “You do not have permission to record activities”. |

| TC_010 | Verify system behavior when the network connection is lost during activity save | • System is operational
• User is logged in
• Network can be simulated to drop | 1. Open **Add New Activity** form.
2. Fill all required fields with valid data.
3. While clicking **Save**, simulate a network outage (e.g., disconnect internet).
4. Observe system response. | Valid activity data (any). | System displays an error message such as “Unable to save activity – please check your connection and try again” and does not create a partial record. The form remains populated so the user can retry after reconnection. |”,

"reasoning": "The user wants 10 test cases derived from the user story and acceptance criteria. However the story is vague: record doctor activities and notes, and the AC is generic. We must still create 10 test cases covering positive, negative, validation. We cannot add assumptions, but we need some specifics. Since limited info, we can create generic test cases: creating a new doctor activity entry, editing, saving, required fields, character limits, duplicate entry, unauthorized user, view recorded notes, delete note, search, etc. Must stay within given story: "record doctor activities and doctor notes". We'll assume UI allows user to add activity, add note, save, view. We'll keep business language. Provide 10 test cases with IDs. Provide preconditions: system operational, user logged in, appropriate permissions. Provide steps, data, expected.

Make sure not to add new requirements beyond what's implied. We can infer fields like activity type, date, description, note text. That's reasonable.

Let's produce."

},

- Response
- messages --role :assistant-content --output generated
- prompt_tokens --input tokens
- completion_tokens --output tokens
- total_tokens --prompt_tokens+ completion_tokens
- "finish_reason": "stop" --entire output is

The screenshot displays a REST client interface within a workspace. The left sidebar shows a collection of API requests, including 'POST S02-GenerateTestcasesfromUserStories'. The main panel shows the details of this request, which is a POST to 'https://api.groq.com/openai/v1/chat/completions'. The request body is in raw format, showing a JSON object with 'role' and 'content' fields. The response is in JSON format, showing a 200 OK status and a JSON object with usage statistics and a 'finish_reason' of 'stop'.

Request Body (raw):

```

{
  "role": "user",
  "content": "I - INSTRUCTION\nConvert the provided User Stories and Acceptance Criteria into detailed, testable Test Cases.\n\nMANDATORY RULES:\n(1) Derive test cases ONLY from the given user stories and acceptance criteria.\n(2) Do NOT add assumptions or new requirements.\n(3) Cover positive, negative, and validation scenarios.\n(4) Write test cases from an end-user perspective.\n(5) Do NOT include implementation or technical details.\n\nCONTEXT\nThese test cases will be used by QA teams for manual and automation testing in an enterprise healthcare application.\n\nProduce high-quality, review-ready test cases."
}

```

Response Body (JSON):

```

{
  "usage": {
    "queue_time": 0.006271167,
    "prompt_tokens": 353,
    "prompt_time": 0.014778293,
    "completion_tokens": 1410,
    "completion_time": 2.9135429200000003,
    "total_tokens": 1763,
    "total_time": 2.9283212130000003,
    "completion_tokens_details": {
      "reasoning_tokens": 202
    }
  },
  "logprobs": null,
  "finish_reason": "stop"
}

```

The response status is 200 OK, with a response time of 5.30 s and a size of 3.49 KB. The response is saved and can be visualized or downloaded.